

Layered Architecture

1. Presentation Manager

Purpose:

- Manages the overall **user interface environment**.
- Provides communication between the user and the software.
- Controls how screens, windows, and pages are displayed.

Responsibilities:

- Manage application screens and windows.
- Control navigation between different pages.
- Manage menus, buttons, forms, and UI components.
- Handle user sessions.
- Control appearance, themes, and layouts.
- Send user actions to the presentation logic layer.

Examples:

- Opening a login screen.
- Moving from login page to dashboard.
- Displaying student registration form.
- Managing menus and toolbars in an application.
- Showing dialog boxes and messages.

Technologies:

- JavaFX Stage and Scene
 - Java Swing JFrame
 - Android Activity
 - Windows Forms
 - WPF
 - Web Browser
 - Flutter Widgets
-

2. Presentation Logic

Purpose:

- Controls the **behavior and processing of the user interface**.
- Defines what happens when a user interacts with the application.

Responsibilities:

- Handle user events.
- Process button clicks.
- Perform input validation.
- Format data before displaying.
- Enable or disable UI controls.
- Display success/error messages.
- Pass valid data to application logic.

Examples:

- Check whether username and password fields are empty.
- Validate email format.
- Display student list in a table.
- Calculate total marks displayed on screen.
- Show error message for invalid input.

Technologies:

- JavaFX Controller
 - Swing ActionListener
 - JavaScript
 - React Components
 - Angular Components
 - ASP.NET MVC View
 - HTML Forms
-

3. Application Logic

Purpose:

- Controls the **flow and coordination of the application**.
- Acts as a bridge between presentation logic and business logic.

Responsibilities:

- Receive requests from presentation layer.
- Control application workflow.
- Call appropriate business services.
- Manage authentication and authorization.
- Handle transactions.
- Coordinate different modules.

Examples:

- Processing login request.
- Managing student admission process.
- Handling online shopping checkout.
- Controlling order processing steps.
- Managing user registration workflow.

Technologies:

- Spring Boot Controller
 - Spring Service Layer
 - ASP.NET Controller
 - Node.js Express
 - Laravel Controller
 - Django Views
 - REST API
-

4. Business Logic

Purpose:

- Contains the **rules and policies of an organization**.
- Performs the main decision-making operations of the software.

Responsibilities:

- Implement business rules.
- Perform calculations.
- Validate business conditions.
- Process data.
- Make decisions based on rules.
- Maintain organization policies.

Examples:

- Check student exam eligibility:
 - Attendance must be greater than 75%.
- Calculate employee salary.
- Calculate bank loan interest.
- Apply product discount rules.
- Check available stock before purchase.

Technologies:

- Java Service Classes
 - Spring Boot Services
 - C# Business Classes
 - .NET Services
 - Python Classes
 - PHP Classes
-

5. Database Logic

Purpose:

- Handles communication between business logic and the database.
- Converts application requests into database operations.

Responsibilities:

- Create database queries.
- Insert records.
- Update records.
- Delete records.
- Retrieve information from database.
- Convert database records into objects.
- Hide database details from business logic.

Examples:

- Insert new student information.
- Search student by roll number.

- Update employee salary details.
- Delete customer record.
- Fetch product information.

Example:

```
SELECT * FROM Student  
WHERE RollNo = 101;
```

Technologies:

- JDBC
 - Hibernate
 - JPA Repository
 - DAO (Data Access Object)
 - Entity Framework
 - ADO.NET
 - ORM Frameworks
-

6. Database Manager

Purpose:

- Manages the actual database system and physical storage of data.
- Responsible for storing, protecting, and retrieving information.

Responsibilities:

- Store data permanently.
- Execute database commands.
- Manage database connections.
- Control transactions.
- Provide security.
- Manage backup and recovery.
- Optimize database performance.

Examples:

- Store student records in tables.
- Execute SQL queries.
- Maintain database files.
- Backup college database.
- Manage multiple user requests.

Technologies:

- MySQL Server
- Oracle Database
- Microsoft SQL Server
- PostgreSQL
- MongoDB
- SQLite